

CONSOLIDATED INTELLIGENCE PLATFORM

AI-Powered Criminal Network Analysis System

Smart India Hackathon 2026 · Problem Statement SIH26189
Theme: Blockchain & Cybersecurity · NCRB Women Safety Division

Team CARIBBEAN

Project Reference Report
Generated 15 September 2026

DATA -> RELATIONSHIP -> PATTERN -> LEAD/PATH -> EVIDENCE -> HUMAN DECISION

This system never becomes: DATA -> AI -> PERSON SCORE -> GUILTY

Why This System Exists

Police forces hold FIRs, call detail records (CDR/IPDR), and financial transaction records in fragmented systems. Organized networks - fraud rings, trafficking rings - deliberately use intermediaries, mule accounts, and burner-SIM rotation to look fragmented on paper even when they are one coordinated operation. The Consolidated Intelligence Platform ingests this fragmented data, resolves identities correctly, builds a relationship graph, surfaces true influencers ("kingpins", not just the busiest phone number), detects hidden structural patterns, and hands investigators evidence-backed, explainable leads - never an automated accusation.

Governing Principle

Every feature in this system is built to respect one directional flow of reasoning, and nothing in the codebase is allowed to skip a step in it:

DATA -> RELATIONSHIP -> PATTERN -> LEAD/PATH -> EVIDENCE -> HUMAN DECISION

It must never collapse into **DATA -> AI -> PERSON SCORE -> GUILTY**. Concretely: every analytical score in this system attaches to a *lead* or a *path* between entities, never to a person as a standalone risk number; every automated finding carries **requires_human_verification: true** and links back to the exact source records that produced it; and disposition (Useful / Already Known / Wrong Person) is always a human action, never something the system decides for itself.

Excluded By Design

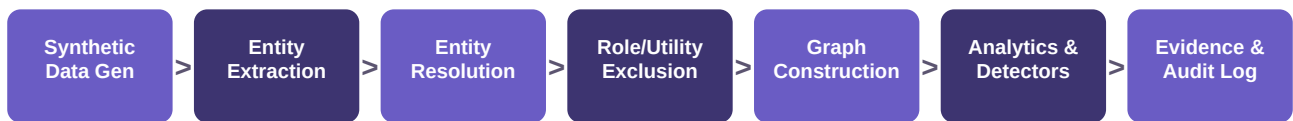
- Face recognition or CCTV analysis
- Dark-web / social-media scraping
- Any per-PERSON risk score framed as predicting guilt or future behaviour
- Deep-learning future-link prediction (GraphSAGE / EvolveGCN-style) - a transparent, weighted recovery score is used instead, precisely because it can be explained to a court
- Real-time streaming ingestion for demo effect - batch processing is realistic for this domain
- Production graph-database migration - documented below as future work only

Technology Stack & Pipeline

Layer	Technology	Role
Backend API	Python 3.12, FastAPI, Uvicorn	REST endpoints, RBAC, request-level audit logging
Data store	SQLite (WAL mode)	Structured records, entities, graph edges, audit chain
NLP / extraction	spaCy (en_core_web_sm) + regex + gazetteer	Entity mentions from free-text FIR narratives
Graph & analytics	NetworkX >= 3.0	Multi-layer graph, Louvain communities, centrality, broker scoring
Frontend	React 18 + TypeScript + Vite	Investigator dashboard SPA
Graph visualization	react-force-graph-2d	Interactive force-directed relationship graph
Auth	PyJWT + passlib(bcrypt)	JWT-based session, case-scoped RBAC
Integrity	hashlib SHA-256 (Python) + Web Crypto SubtleCrypto (browser)	Independent dual verification of the audit hash-chain

Ingestion & Analysis Pipeline

Every run executes these stages in order (idempotent, re-runnable end to end via one command):



- **Synthetic data generation** - produces FIR text, CDR call records, and bank transaction records with a fixed random seed, plus a machine-checkable `ground_truth.json` checklist of every planted scenario.
- **Entity extraction** - spaCy NER, a role-keyword regex backstop (Accused/Witness/Officer naming), a curated place-name gazetteer, and structured-identifier regex (phone/account/vehicle), each mention tagged with its extraction method and an honestly-labelled confidence tier.
- **Entity resolution** - union-find over mentions; auto-merge only on name similarity *plus* a shared hard identifier in the same source record; unconfirmable name-similar mentions are grouped into one human review cluster.
- **Role/utility exclusion** - flags officials (by extracted ROLE field, never by string-matching a name) and structurally-detected utility numbers (high in-degree, near-zero out-degree), removed from the analysis graph before any ranking is computed.
- **Graph construction** - typed nodes (PERSON/PHONE/ACCOUNT/VEHICLE/LOCATION/ORGANIZATION/CASE) and typed, timestamped edges carrying an `epistemic_status` (OBSERVED / NLP_EXTRACTED / INFERRED / RECOVERED).
- **Analytics & detectors** - degree, betweenness, PageRank, Louvain communities, cross-community broker scoring, and five hidden-structure detectors (below).
- **Evidence & audit** - every detector hit becomes a Lead with concrete signals and source record IDs; every query and disposition is appended to the SHA-256 hash-chained audit log.

Planted Ground-Truth Scenarios

Two cases are generated: **C001 - Fraud Ring Alpha** (financial fraud) and **C002 - Missing Persons: Sonepur Corridor** (the Women-Safety flagship scenario). Eleven scenarios are deliberately planted and independently re-verified against the pipeline's actual output - the table below is generated from the current run, not written by hand.

#	Scenario	Planted detail	Tests it against
1	Name collision	Two 'Rajesh Kumar's: 9810000001 / 9810000002	Must NOT auto-merge on name alone
2	Alias merge	'Sunita Devi' / 'Sunita D.' share phone 9810000003	Must merge via shared hard identifier
3	Official exclusion	Inspector Alok Sharma, 15 FIR mentions, phone 9810000090	Must never top any ranking
4	Utility exclusion	Toll-free 18001800001, callee-only, 59 distinct callers	Must never top any ranking
5	Call-before-transfer	25-minute call then a Rs. 8.5L transfer, same two people	Cross-source temporal motif detector
6	Burner-SIM rotation	9810000020 silent, 9810000021 activates 1 day later, 70% contact overlap	Burner-rotation detector
7	Mule layering	40 senders -> 6 layer-1 accounts -> 2 layer-2 accounts	Fan-in/fan-out recursion-depth detector
8	Broker bridge	One low-volume account linking two dense, unrelated 8-node communities	Cross-community broker scoring
9	Trafficking chain	Recruiter -> 4 victims (+2 victim-victim edges), 1 transporter, 1 receiver in a 4-node receiver cluster	Women-Safety two-method bridge detection
10	Repeat location	'Sonepur Junction' named in 3 independent FIRs tied to different entities	Repeat-location signal
11	Mundane recurrence	'Sonepur Police Station' files many unrelated FIRs (expected, not flagged)	Negative control for the repeat-location signal

Deliberately absent: no hand-crafted "hidden edge" is planted for missing-link testing - that capability is validated by masking a random slice of the real generated edges instead (Section 7), so the evaluation is never checking against data that was made to be found.

Extraction & Union-Find Resolution

Why a rule-based backstop, not NER alone

A general-purpose small NER model genuinely misses or truncates names in short, FIR-style sentences - verified during development, not a hypothetical risk (it dropped a full name in one sentence and truncated another to a single token). Two backstops compensate: a role-keyword regex keyed to words like "Accused", "Witness", "Investigating Officer" (which also strips rank prefixes like "Inspector" before treating the remainder as a name), and a small curated gazetteer for Indian place names the small NER model handles poorly.

Resolution rule

PERSON mentions are **never** merged on name similarity alone. Two name-similar mentions merge only when each also carries a hard identifier (phone/account/vehicle) extracted from its own source record, and those identifiers resolve to the same value. Three outcomes follow from this:

Outcome	Condition	Example from this dataset
Confirmed merge	Names similar AND identifiers match	'Sunita Devi' + 'Sunita D.' -> one PERSON entity
Confirmed distinct	Names similar BUT identifiers differ	Two 'Rajesh Kumar's -> two PERSON entities, no review needed
Ambiguous -> review	Names similar, identifier missing on at least one side	15 mentions of 'Alok Sharma' -> ONE review cluster (not 15, not C(15,2) pairs)

Ambiguous mentions are grouped with a two-pass union-find so that N mentions of one unconfirmable name produce exactly one review-queue cluster, never one row per pair - a naive pairwise comparison would have produced up to $C(15,2) = 105$ duplicate rows for the officer alone.

Role typing and exclusion

An entity is flagged official from its extracted *role* field, never by checking whether its own name string contains a keyword like "inspector" (a person's name will never literally contain that word). The flag then propagates to any phone/account mentioned in the same FIR record as that official - their own duty phone, not a suspect's. Utility numbers are detected purely structurally: high in-degree, near-zero out-degree in the call graph. Both exclusion sets are computed once and every ranking function (degree, betweenness, PageRank, Louvain, broker score) builds its working graph from the same excluded-node-free subgraph, so an exclusion cannot be applied in some rankings and missed in others.

Detectors & Live Results

Every detector below was run against the current dataset; the counts are from that live run.

Detector	Method	Live result
Burner-SIM rotation	Jaccard contact-set overlap + activation-gap, checked in both chronological directions (never inferred from string sort order), with minimum absolute shared-contact count as well as a ratio	1 rotation found: 9810000020 -> 9810000021, 7 shared contacts, Jaccard 0.70, 1-day gap
Mule-account layering	Sliding-window fan-in detection, then recursion-depth scoring of the aggregate-then-forward chain (not a circular-flow rule)	2 funnels found: 6 layer-1 accounts feeding 2 layer-2 accounts, recursion depth 2
Cross-source temporal motif	A call between two phones followed by a large transfer (amount threshold = mean + 3 std. dev. of this run's own transaction distribution) between the accounts those phones are linked to via FIR co-occurrence	1 motif found: 25.0-minute gap, Rs. 850,000 transfer, threshold Rs. 239,483
Cross-community broker	Count of distinct Louvain communities a node's neighbours span, excluding its own	2 broker-bridge leads (the planted low-volume broker account and its counterpart on the bridged side)

Every finding becomes an evidence-backed Lead

The Evidence Engine converts each detector hit into a Lead object: lead_type, severity, the specific entities involved, a human-readable why-flagged summary, structured signal values, and the exact source record IDs. **requires human verification** is true on every single lead in this system - there is no code path that produces a finding framed as a conclusion.

Lead type	Count (this run)
BROKER BRIDGE	2
BURNER ROTATION	1
MULE LAYERING	2
CALL BEFORE TRANSFER	1
WOMEN SAFETY RECRUITER	4
WOMEN SAFETY TRANSPORTER	8
REPEAT LOCATION	1
TOTAL	19

Women-Safety / Trafficking-Network View

This is **not a separate detection model**. It re-labels and filters the SAME community-detection and broker-scoring machinery used for the general financial "kingpin" analysis above, applied to the person/phone network. The pitch is literally true in the implementation: the same mechanism that surfaces a financial intermediary surfaces a trafficking one - both call `detect_transporter_candidates()` over the same analysis graph.

A documented limitation, engineered around rather than hidden

Louvain community detection does not reliably split a single small, sparse recruiter -> transporter -> receiver chain into separate communities absent a second, competing dense structure - it tends to merge the whole weakly-connected chain into one community. This was verified directly: in this dataset, the recruiter, all four victims, the transporter and the receiver-side all land in the *same* Louvain community. So two independent methods are run and merged, each hit tagged with which method found it:

Method	Definition	Result on this dataset
COMMUNITY_BRIDGE	A node whose direct neighbours span ≥ 2 distinct Louvain communities	0 hits for the trafficking chain (confirms the documented limitation above)
STRUCTURAL_BRIDGE_PATTERN (primary)	A low-degree (≤ 3), low-call-volume (≤ 3) node with one neighbour in a known recruiter's contact set AND another neighbour whose own degree is comparatively high (≥ 4)	Correctly identifies the planted transporter (phone 9820000002)

The recruiter heuristic is deliberately generic

Fan-out to ≥ 3 low-degree contacts within one community also flags unrelated hub structures - in this dataset it correctly also flags both burner-rotation phones. This is documented as expected, not a bug: narrowing the heuristic until it only matches the planted trafficking case would be overfitting to synthetic data instead of building a general detector. Every candidate carries `requires_human_verification=true` for exactly this reason.

Repeat-location signal

The same real-world location named across ≥ 3 independently-sourced records is investigatively meaningful. 'Sonepur Junction' is correctly flagged (3 independent FIRs: a missing-person report, unrelated surveillance intelligence, and a separate case file, tied to different entities). 'Sonepur Police Station' - which recurs across many more FIRs simply because it is the filing station - is correctly *not* flagged, because the extraction pipeline uses a separate police-station gazetteer that tags it ORGANIZATION rather than LOCATION, so it never enters this signal's scope at all.

The connector-card motif, reused deliberately

The recruiter -> transporter -> receiver-side chain is structurally the same shape as the product's own connector-card design motif (used elsewhere for top-level case stats), so the frontend renders it literally as three connected cards showing the real entity IDs and their real supporting metrics - a content-driven design choice, not decoration, and the UI copy says so explicitly next to the chain.

Honest, Masked-Edge Evaluation

Framed strictly as "verify this relationship may already exist" - never as a forecast. The score blends two normalized (0-1), documented-weight components:

Component	Weight	Definition
Structural similarity	0.6	Jaccard overlap of the two candidate nodes' neighbour sets
Cross-source support	0.4	Of their common neighbours, the fraction corroborated by more than one distinct relationship type (e.g. a shared call contact AND a shared transfer counterparty) rather than one repeated source

Output tiers: HIGH (≥ 0.66), MEDIUM (≥ 0.40), LOW (below). No planted hidden edge exists anywhere in the dataset for this test - validation instead masks a random 20% of the REAL generated edges, runs recovery blind to them, and reports recall against the actual held-out set, computed at run time:

Metric	Value (this run)
Edges masked (20%)	48
Candidate pairs scored	426
Recall@10	0.0208
Recall@20	0.0417
Recall@50	0.0417

These numbers are reported honestly rather than tuned up. They are genuinely low, and the reason is structural rather than a scoring-method defect: a large share of the masked edges are single-degree pendant edges (for example a mule sender with exactly one transaction to its layer-1 account), which carry zero common-neighbour signal for any similarity-based method to recover. This is disclosed here and in the running Self-Evaluation dashboard rather than adjusted after the fact - per the project's own rule that this metric must never be hard-coded or improved post-hoc.

Tamper-Evident Audit Chain & RBAC

SHA-256 hash chain

Every analytical action - a graph query, lead generation, a disposition, a review-queue resolution, a full pipeline run - is appended to an audit_log table as hash = SHA-256(prev_hash + "|" + payload_raw). payload_raw is a plain, deterministic string (not a JSON re-serialization), stored verbatim, specifically so the browser does not need to reimplement Python's serialization to reproduce the same hash.

Verified twice, independently

The backend recomputes the entire chain in Python on request. The browser *independently* re-hashes the exact same raw strings via the Web Crypto SubtleCrypto API and compares - two separate implementations with no shared trust. A live tamper-then-restore cycle is built into the Audit Chain tab (admin role only): mutating one entry's stored reason (without recomputing its hash) is immediately flagged BROKEN by both the Python check and the browser check; restoring the original text makes both report CHAIN VALID again. This was demonstrated end-to-end in the running UI, not just asserted in a unit test.

RBAC & case scoping

Role	Access
investigator1 / investigator2	Case-scoped - only assigned cases are visible; enforced server-side via a dependency check, not just hidden in the UI
admin1	All cases, plus the audit tamper/restore demo tools

Every case-scoped read (graph, leads, analytics, women-safety view) requires a non-empty *reason for this query*, which is itself logged to the audit chain - the case-selection screen refuses to proceed without one.

Frontend & Design System

React + TypeScript + Vite, styled to a deliberately distinctive dark Web3-product visual language (not a generic SaaS admin theme): a single rounded panel on a soft lavender page background, a radial violet glow, faint orbit-ring outlines, JetBrains Mono for data-like text (nav labels, IDs, stat numbers) and Space Grotesk for prose, and a recurring rounded "connector-card" motif linked by a dotted line - used for top-level case stats and, meaningfully, for the Women-Safety recruiter->transporter->receiver chain.

Page	What it shows
Case Selection	Assigned cases + mandatory audit-logged reason-for-query field
Overview	Connector-card case stats, entity-type mix, recent high-severity leads
Graph Explorer	Force-directed graph; colour = entity type; red ring = excluded (official/utility); dashed edge = NLP-extracted; click any node for its full evidence trail (FIR text, call records, transactions)
Leads	Filterable lead list with why-flagged signals, source record IDs, and disposition buttons (Useful / Already Known / Wrong Person)
Review Queue	Identity-resolution clusters awaiting human merge/keep-separate/escalate decisions
Women Safety (flagship)	Recruiter and transporter candidates with method provenance, the connector-card chain visualization, and the repeat-location signal list
Self-Evaluation	Live resolution/exclusion/detector checks, NER score summary, masked-edge recall, and per-stage pipeline timings - all computed from the current run
Audit Chain	Full hash-chain listing with dual backend/browser verification and the admin tamper-restore demo

Automated Tests & Self-Evaluation

13 automated pytest cases assert every planted ground-truth scenario end to end against the real pipeline output - not a smoke test that it merely runs without crashing. All 13 currently pass.

Test	Result
test_name_collision_kept_separate	PASS
test_alias_spellings_merged	PASS
test_officer_excluded_and_never_tops_ranking	PASS
test_utility_excluded_and_never_tops_ranking	PASS
test_burner_rotation_detected	PASS
test_mule_layering_detected	PASS
test_call_before_transfer_detected	PASS
test_bridgeBroker_detected	PASS
test_women_safety_recruiter_and_transporter_found	PASS
test_repeat_location_signal	PASS
test_mundane_station_not_flagged_as_repeat_location	PASS
test_audit_chain_detects_and_recovers_from_tampering	PASS
test_masked_edge_recovery_runs_and_reports_recall	PASS

Self-evaluation dashboard (live, from the running app)

Check	Result
name_collision_kept_separate	PASS
alias_spellings_merged	PASS
officer_phone_flagged_official	PASS
officer_never_tops_ranking	PASS
officer_excluded_from_analysis_graph	PASS
utility_number_flagged_utility	PASS
utility_never_tops_ranking	PASS
utility_excluded_from_analysis_graph	PASS
burner_rotation_detected	PASS
mule_layering_detected	PASS
call_before_transfer_detected	PASS
women_safety_recruiter_found	PASS
women_safety_transporter_found (STRUCTURAL_BRIDGE_PATH)	PASS

NER / extraction score summary (live)

Extraction method	Mentions	Avg. score
REGEX (structured identifiers)	18	0.95
ROLE_KEYWORD (role-labelled names)	22	0.85
GAZETTEER (curated places/stations)	4	0.80
SPACY_NER (supplementary)	1	0.60

Scores are fixed, honestly-labelled tiers per extraction method, not a fabricated per-entity confidence the underlying model doesn't actually expose.

Stage timings (most recent full pipeline run)

Stage	Seconds
data generation	0.0392
entity extraction	0.5492
entity resolution	0.0165
role utility exclusion	0.0146
graph construction	0.0176
total	0.6472

Record counts	Value
fir records	35
cdr records	179
transaction records	136
mentions extracted	45
entities created	228
review clusters	1
graph edges	357

Limitations & Future Work

- **SQLite is dev-grade.** A production graph database (e.g. Neo4j) is deliberately documented as future work rather than built now, per the project's own scope decision to prioritize a reliably demoable transparent system over infrastructure that adds risk without adding explainability.
- **The Women-Safety view is intentionally cross-case,** not filtered to one case, because trafficking-network analysis is meant to run across the whole dataset - this also means the deliberately generic recruiter heuristic's expected false positives (e.g. a burner-rotation phone) surface there regardless of which case is open.
- **Missing-link recall is genuinely low** on this sparse synthetic graph (Section 7) - a larger, denser real dataset would give the structural-similarity method more common-neighbour signal to work with.
- **Case context in the frontend is session-scoped** (sessionStorage), by design - it survives an accidental refresh within the same login session but not a new session, since a case-query reason is meant to be tied to that specific audit-logged session.
- **No real-time ingestion.** Batch processing only, which is realistic for this domain and keeps the system demoable without a live data feed.

Natural next steps for continued development

- Swap SQLite for a production-grade graph database once the schema stabilizes against real data.
- Extend the synthetic-data generator (or plug in real anonymized data) to stress-test recall on a denser graph.
- Add per-case filtering as an option on the Women-Safety view for investigators who want a narrower slice, while keeping the cross-case view as the default.
- Layer in a proper user-management screen (currently three hardcoded demo accounts) if this moves beyond a hackathon prototype.

Exact Commands

1. Backend

```
cd backend
venv\Scripts\activate
python -m app.pipeline # regenerates data, extracts, resolves, builds the graph
python -m uvicorn app.main:app --port 8000
```

2. Frontend

```
cd frontend
npm install # first time only
npm run dev -- --port 5173
```

3. Open

Navigate to **http://localhost:5173**.

Account	Password	Access
investigator1	investigator1pass	Case C001 - Fraud Ring Alpha
investigator2	investigator2pass	Case C002 - Missing Persons - Sonapur Corridor
admin1	admin1pass	Both cases + audit tamper/restore demo tools

4. Run the automated test suite

```
cd backend
venv\Scripts\activate
python -m pytest tests/ -v
```

Note: the test suite regenerates the synthetic dataset with a fixed random seed before running (reset_data=True), so re-running it after a demo session will reset any dispositions or review decisions made through the UI back to the clean planted state.

Repository Structure

```
backend/app/  
  data_generation/generator.py - synthetic FIR/CDR/transaction data + ground truth  
  extraction/ - ner.py, regex_patterns.py, gazetteer.py, extractor.py  
  resolution/ - union_find.py, resolver.py  
  graph/ - builder.py, exclusion.py, analytics.py, case_view.py  
  detectors/ - burner_sim.py, mule_layering.py, temporal_motif.py, women_safety.py  
  recovery/missing_link.py - structural + cross-source recovery scoring  
  evidence/ - engine.py (leads), lookup.py (entity evidence detail)  
  audit/chain.py - SHA-256 hash-chain audit log  
  auth/rbac.py - JWT auth, roles, case assignments  
  evaluation/self_eval.py - self-evaluation dashboard data  
  api/routes.py - all FastAPI endpoints  
  pipeline.py - orchestrates the full run, records stage timings  
backend/tests/test_ground_truth.py - the 13 automated ground-truth assertions  
  
frontend/src/  
  pages/ - Login, CaseSelect, Dashboard, Overview, GraphExplorer, Leads, ReviewQueue,  
  WomenSafety, SelfEvaluation, AuditChain  
  context/ - AuthContext, CaseContext  
  api/client.ts - typed API client  
  components/common.tsx - Eyebrow, badges, ConnectorChain, StatCard  
  styles/theme.css - the full design-system token set
```

Generated from a live pipeline run on 2026-09-15T08:52:19Z. Every figure in this report traces to that run's actual output (backend/data/last_pipeline_run.json and the self-evaluation endpoint) or to the pytest suite's own result - nothing here is a projected or hoped-for number.